

Introdução à Computação montando uma biblioteca em C

Alexandre Rademaker

montando uma biblioteca I

Temos uma coleção de funções para manipular listas encadeadas.

Como transforma-las em uma biblioteca? Como compartilha-las entre vários programas que precisem de listas encadeadas?

Como compilar a biblioteca separadamente dos programas que a utilizem?

O arquivo list.h

list/list.h

```
1 typedef struct node t_node;
2
3 int      list_nth(t_node *list, int n);
4 ...
5
6 t_node* list_insert(int element, t_node* list);
7 t_node* list_reverse(t_node *a);
8 t_node* list_create(int arr[], int N);
9 t_node* list_append(t_node *a, t_node *b);
10 ...
11
12 void     list_nconc(t_node *a, t_node *b);
13 void     list_print(t_node *list);
14 ...
```

Neste arquivo vamos **declarar** os nomes (símbolos).

O arquivo list.c

list/list.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "list.h"
4
5 typedef struct node {
6     int number;
7     struct node *next;
8 } t_node;
9
10 ...
```

Neste arquivo vamos **definir** a estrutura e as funções.

O arquivo test.c |

list/list-test.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "list.h"
4
5 int main(int argc, char *argv[]) {
6     int tmp[] = {10, 20, 30};
7     t_node *a = list_create(tmp, 3);
8     list_print(a);
9     t_node *ac = list_copy(a);
10    list_print(ac);
11    list_print(list_reverse(ac));
12    t_node *s = list_append(a, a);
13    list_print(s);
14    list_nconc(ac, ac);
15    list_print(ac); // Ops!
16 }
```

O arquivo test.c II

```
1 CL-USER> (setf *print-circle* t)
2 T
3 CL-USER> (let ((a (list 1 2 3))
4               (b (list 10 20 30)))
5           (values a b (nconc a a)))
6 #1=(1 2 3 . #1#)
7 (10 20 30)
8 #1=(1 2 3 . #1#)
```

Qual o bug?! Testes que faltam!?

a compilação I

Imagine que `list-test.c` é um programa que use nossa biblioteca. Poderiam haver outros.

Queremos separar a compilação da biblioteca da compilação dos programas que usam a biblioteca.

Afinal, em algum momento nossa biblioteca ficará **estável** e não irá sofrer alterações.

a compilação II

Compilando a biblioteca geramos o `list.o`:

```
clang -c -Wall list.c
```

Compilando o programa `list-test.c` que usa a biblioteca e incorporando o binário da biblioteca no arquivo binário (executável) final.

```
clang -Wall list-test.c list.o -o test
```


a compilação III

Também podemos não incorporar o binário da biblioteca no executável final, criando um **shared library**, várias outras podem ser encontradas no `/usr/local/lib`.

```
clang -shared list.o -o list.so
```

seguido de

```
clang -Wall list-test.c list.so -o test
```

a compilação IV

Mas podemos usar make para nos ajudar!

O make lê um arquivo Makefile e determina: 1) que comandos deve executar; 2) em qual ordem eles devem ser executados.

Considere o comando:

```
clang -c -Wall list.c
```

A entrada é o arquivo `list.c` e ele produz o arquivo `list.o`, então o make saberá que rodar este comando só será necessário se a **data da última alteração** de `list.c` for mais recente que a **data de criação** de `list.o`.

a compilação V

Makefile

```
1 OPT=-g -Wall ...
2
3 all: test
4
5 list.o: list.c
6     clang -c ${OPT} list.c
7
8 list.so: list.o
9     clang -shared list.o -o list.so
10
11 test: list.so list.h list-test.c
12     clang ${OPT} list-test.c list.so -o test
13
14 clean:
15     rm test list.o list.so
```

Exemplos extras

Arquivos em `src/` relacionados ao tema desta semana, para estudo complementar (não foram apresentados em aula):

- ▶ `arrays-0.c`, `arrays-1.c`, `arrays-2.c`, `arrays-3.c`,
`arrays-4.c`
- ▶ `matriz.c`
- ▶ `pointers-of-functions.c`, `pointers-of-pointers.c`
- ▶ `list/list-glib.c` (variante com GLib)
- ▶ `test.txt` (entrada de exemplo)